# MapReduce数据挖掘基础算法(III)




# 摘要



- 频繁项集挖掘算法
- 应用案例




# 频繁项集挖掘



- 关联规则或频繁项集
  - 频繁项可以看作是两个或多个对象的“亲密”程度，如果同时出现的次数很多，那么这两个或多个对象可以认为是高关联性的。当这些高关联性对象的项集出现次数满足一定阈值时即称其为频繁项。
- 经典实例：购物篮分析
  - 啤酒和尿布



# 频繁项集



- m个项的集合： $I = \{I_1, I_2, \dots, I_m\}$
- n个事务的数据库： $D = \{T_1, T_2, \dots, T_n\}$，其中$T_i$是$I$的非空子集。
- 关联规则： $X \Rightarrow Y$ ，其中$X$，$Y$是$I$的子集，并且$X \cap Y = \emptyset$
- 关联规则的$X \Rightarrow Y$的支持度是指$D$中事务包含$X \cup Y$的百分比，置信度是指包含$X$的事务中同时包含Y的百分比，也就是条件概率$P(Y|X)$
- 如果某一规则$X \Rightarrow Y$的置信度和支持度都高于某个阈值，那么可以认为$X$，$Y$具有关联性。



# 频繁项集



- 把满足上述条件的$X$，$Y$组成一个项集$A$，则可以认为，数据库$D$中包含$A$的事务超过了某个最小支持度。而所谓频繁项集挖掘，就是将所有满足项数为某个固定整数$k$，且支持度高于某阈值的项集计算出来。
- 经典算法：Apriori，FP-growth，SON等。

# Apriori算法



- 通过多轮迭代的方法来逐步挖掘频繁项集
- 先验原理：
  - 频繁项集的任何非空子集都是频繁的。
  - 非频繁项集的任何超集都是非频繁的。

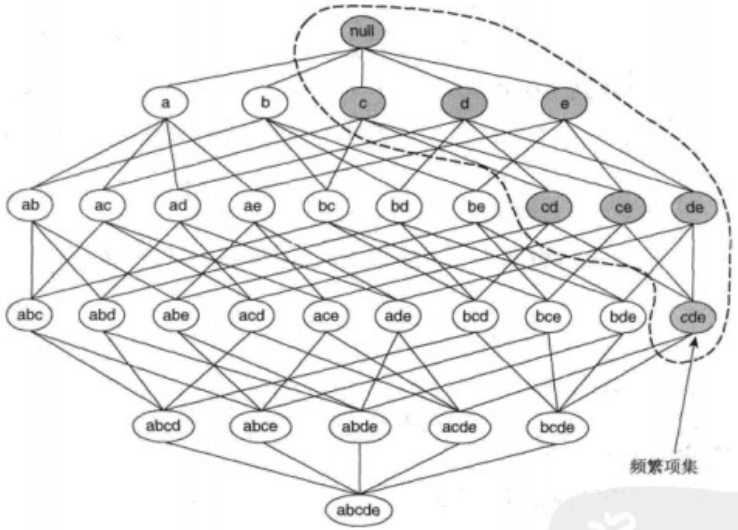



图 6-3　先验原理的图示。如果{c, d, e}是频繁的，则它的所有子集也是频繁的

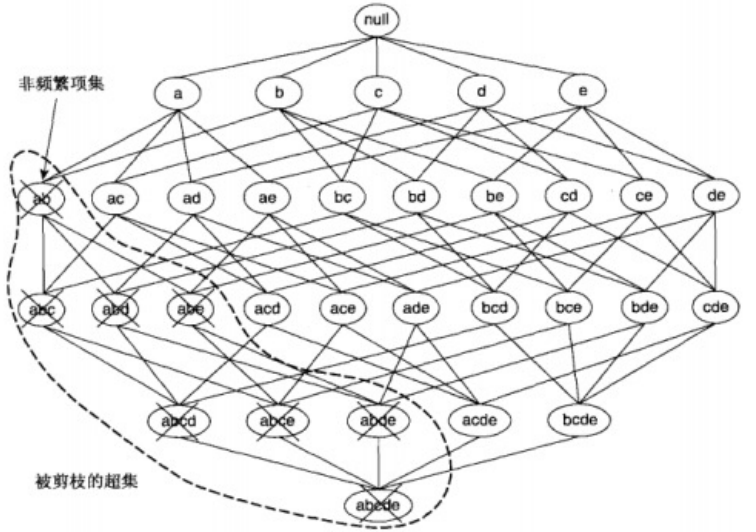



图 6-4　基于支持度的剪枝的图示。如果{a,b}是非频繁的，则它的所有超集也是非频繁的



# Apriori算法



$L_1$={frequent 1-itemsets};

For (k=2; $L_{k-1} \neq$ null; k++} do begin　　//多轮迭代过程

　$C_k$=apriori-gen($L_{k-1}$);　　//候选k-项集

　for each transaction t ∈ D do begin

　　$C_t = \text{subset}(C_k, t)$;　　//生成子集

　　for each candidate c ∈ $C_t$ do

　　c.count ++;　　//计算实际的支持度

　　end

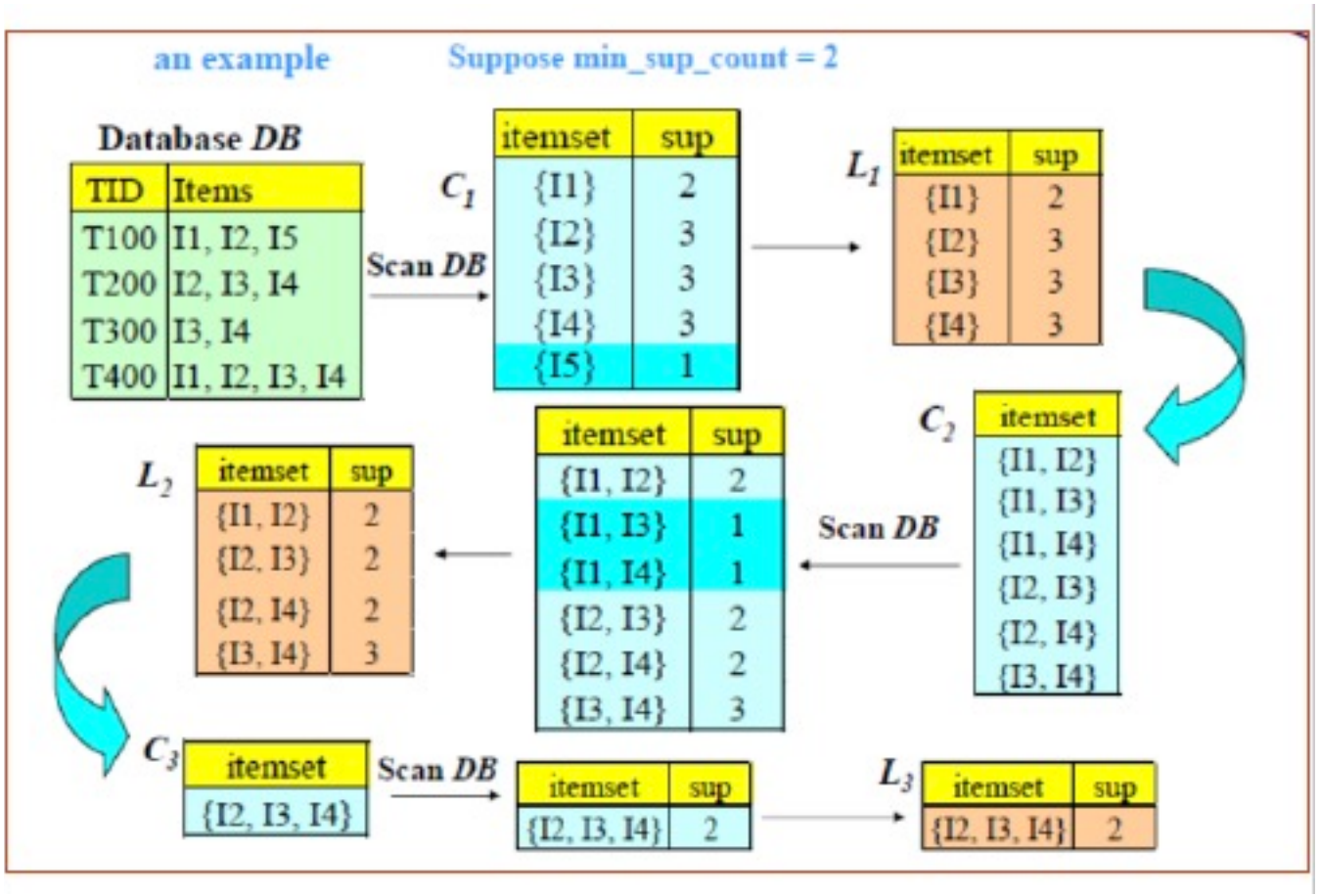
　　$L_k$={ c= $C_k$ | c.count ≥ minsup }

　end

Answer = $U_k L_k$

# Apriori算法




an example
Suppose min_sup_count = 2
Database DB
TID Items
T100 I1, I2, I5
T200 I2, I3, I4
T300 I3, I4
T400 I1, I2, I3, I4
Scan DB
C1
itemset sup
{I1} 2
{I2} 3
{I3} 3
{I4} 3
{I5} 1
L1
itemset sup
{I1} 2
{I2} 3
{I3} 3
{I4} 3
C2
itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I2, I3}
{I2, I4}
{I3, I4}
Scan DB
itemset sup
{I1, I2} 2
{I1, I3} 1
{I1, I4} 1
{I2, I3} 2
{I2, I4} 2
{I3, I4} 3
L2
itemset sup
{I1, I2} 2
{I2, I3} 2
{I2, I4} 2
{I3, I4} 3
C3
itemset
{I2, I3, I4}
Scan DB
itemset sup
{I2, I3, I4} 2
L3
itemset sup
{I2, I3, I4} 2

# Apriori并行化算法设计



- 主进程扫描事务数据库，从原始数据集中产生候选1-项集
- 由候选1-项集与原始数据集进行对比，算出每个项集的支持度，这些支持度与程序中给定的支持度进行对比，得出频繁1-项集
- 由频繁1-项集产生候选2-项集，并通过与原始数据集比较得到频繁2-项集
- 逐次迭代直到产生候选k-项集，候选k-项集与原始数据集对比，如果存在频繁k-项集，则继续迭代执行；如果不存在，则最终得到频繁(k-1)-项集



# Apriori并行化算法设计



- 第一轮迭代：计算频繁1-项集
- Map阶段
  - 输入key：事务数据库每行首字符相对于文本文件的首地址偏移；输入value：存储在事务数据库文件中的一行数据
  - 输出key：候选1-项集　输出value：1

```
Map Task:
    For each transaction t in Si
      Map(line offset, t)
        For each item I in t
            Emit (I, 1);
        End For each
      End Map()
    End For each
End Map
```



# Apriori并行化算法设计



- Reduce阶段
  - 输出key：频繁1-项集；输出value：出现的次数

---

```
Reduce Task:
    Reduce(key2, value2)
        Sum = 0;
    While (value2.hasNext())
        Sum += value2.hasNext();
     End While
     If (Sum>=Minsup)
         Emit (key2, sum)            //得到频繁1-项集
     End If
End Reduce
```



# Apriori并行化算法设计



- 第k轮MapReduce迭代：计算k-频繁项集
- Map阶段
  - 输入<key,value>对为：事务数据分片中的每个事务数据。这个阶段会从频繁(k-1)-项集得到候选k-项集，频繁(k-1)-项集存放在Distributed Cache中以便每个Map节点共享读取和使用。为了产生候选k-项集，包括连接和剪枝过程。
  - 输出key为候选k-项集，value为其出现的次数。



# Apriori并行化算法设计



Map Task:

Read $L_{k-1}$ from HDFS file

$C_k$ = ap_gen($L_{k-1}$)　　//self join连接过程

For each transaction t in Si

Map(line offset, t)

$C_t$ = subset ($C_k$ , t)　　//获取在原始事务数据中出现的候选项集

For each item c in $C_t$

Emit (c, 1);

End For each

End Map()

End For each

End Map



# Apriori并行化算法设计



- Reduce阶段
  - 输出key：频繁k-项集；输出value：出现的次数

---

```
Reduce Task:
    Reduce(key2, value2)
        Sum = 0;
    While (value2.hasNext())
        Sum += value2.hasNext();
     End While
     If (Sum>=Minsup)
         Emit (key2, sum)            //得到频繁k-项集
     End If
End Reduce
```

# 基于子集求取的频繁项集挖掘算法



- 一个频繁项集$F$必然是$D$中某个事务的子集，即只要根据事务数据本身就可以计算频繁项集。
- 假设数据库为$D$，事务为$T$，项集大小为$k$
  - 首先扫描数据库，对数据库中的每一个事务做如下操作：将该事务所有大小为$k$的子集求出来(Mapper)
  - 然后，统计输出所有子集的个数，如果某个子集的个数超过了某一阈值$S$，那么就可以认为这个子集是频繁项集，将所有这样的子集输出即可(Reducer)



# 基于子集求取的频繁项集挖掘算法



- Mapper

```
Class MiningMapper{
    map(T){  //T为数据库中的一个事务
     /** 求出T所有大小为k的子集 **/
     /** 若T的项数小于k，则子集数为0 **/
     Subsets = FindSubsets(T, k);
     for (i=0; i<Subsets.size(); i++)
          subset = Subsets.get(i)
          emit(subset, 1)
     }
}
```



# 基于子集求取的频繁项集挖掘算法



- Reducer

```
Class MiningReducer{
    reduce(key, value_list){
        sum = 0;
        while(value_list.hasNext())
            sum += value_list.next().get();
        if(sum > minSupport)
            emit(key, sum);
    }
}
```



# SON算法



- 有限扫描：最多两遍之内发现全部或者大部分频繁项集。只利用数据抽样样本而不是全部数据集进行计算。
- 基本思路：先采样部分样本，在其上运行Apriori算法，并调整相应支持度阈值。采样得到的子数据集直接存入主存，之后扫描数据自己不用再次进行I/O操作。
  - 例如选择1%的样本，支持度阈值为s/100
- 抽样可能导致完整数据集上的频繁项在样本中不频繁(false negative)，样本中频繁但在整个数据集上不频繁(false positive)



# SON算法



- SON算法同时避免了false negatives和false positives，所带来的代价是需要两次扫描。
- 基本思想：将输入文件划分成1/p个块(p为样本占全部数据集的比例)，将每个文件块作为一个样本，并执行Apriori算法。同样适用p*s作为其阈值。将每个块找到的频繁项集放到磁盘上。



# SON算法



- 第一步：找到局部频繁项集
  - 一旦所有的块按此方式被处理，将那些在一个或多个块中被选中的频繁项集收集起来作为候选频繁项集。注意，如果一个项集在所有的块中都不是频繁项集，即它在每个块中的支持度都低于p*s。因为块数为1/p，所以，整体的支持度也就低于(1/p)*p*s=s。这样，每个频繁项集必然会出现在至少一个块的频繁项集中，于是，我们可以确定，真正的频繁项一定全在候选频繁项集中，因此这里没有false negatives。
- 第二步：找出全局频繁项集
  - 计算所有的候选频繁项集，选择那些支持度至少为s的作为频繁项集。



# SON算法并行化



- 第一个Map 函数：使用子集，并在子集上使用Apriori算法找出每个项集的频繁度。将支持度从s降为p*s，输出(F,1)，F为该样本的频繁项集。
- 第一个Reduce函数：产生那些出现一次或多次的key（即项集），输出候选项集。
- 第二个Map函数：接受第一个Reduce函数的所有输出和输入数据文件的一部分，每个Map任务计算每个候选项集在当前数据集上出现的次数，输出为(C,v)组成的集合，其中C是一个候选集，v是C在本Map任务所分配数据上的支持度。
- 第二个Reduce函数：将分配的项集作为Key，并将关联的值求和。最终得到每个Reduce任务所分配的每个项集在整个数据集上的全部支持度。那些值求和后不低于s的项集是整个数据集上的频繁项集，Reduce任务会将这些项集及其计数值输出。




# 应用领域



- 购物篮分析
- 信用卡交易分析
- 电话呼叫模式分析
- 医疗保险欺诈识别
- 电信服务交易分析
- 大型在线零售商的每日/周交易分析
- ……

# 基于关联规则挖掘的图书推荐算法



- 基于Apriori基本算法实现
  - 采用类似单词计数的过程并行扫描数据库，找出满足最小支持度的1-频繁项集$L_1$
  - $L_1$通过自身连接产生2-候选项集$C_2$，采用候选项集支持度的并行化统计方法统计$C_2$的支持度，行成2-频繁项集$L_2$；以此类推，$L_2$产生$L_3$，如此迭代循环，直到完成k-频繁项集的计算。



# 基于关联规则挖掘的图书推荐算法



- 数据获取：豆瓣读书网站API
  - 获取书籍信息
  - 获取特定书籍的所有评论
- 算法实现：
  - 计算每本图书的支持度
  - 统计2-频繁项集中每个元组的支持度
  - 计算每个2-频繁项集元组中图书A到图书B的置信度

# 图书推荐系统



喜欢读"百年孤独"的人也喜欢 · · · · · ·




霍乱时期的爱情




月亮和六便士




围城




小王子



没有人给他写信的上校




白夜行




不能承受的生命之轻




一桩事先张扬的凶杀案



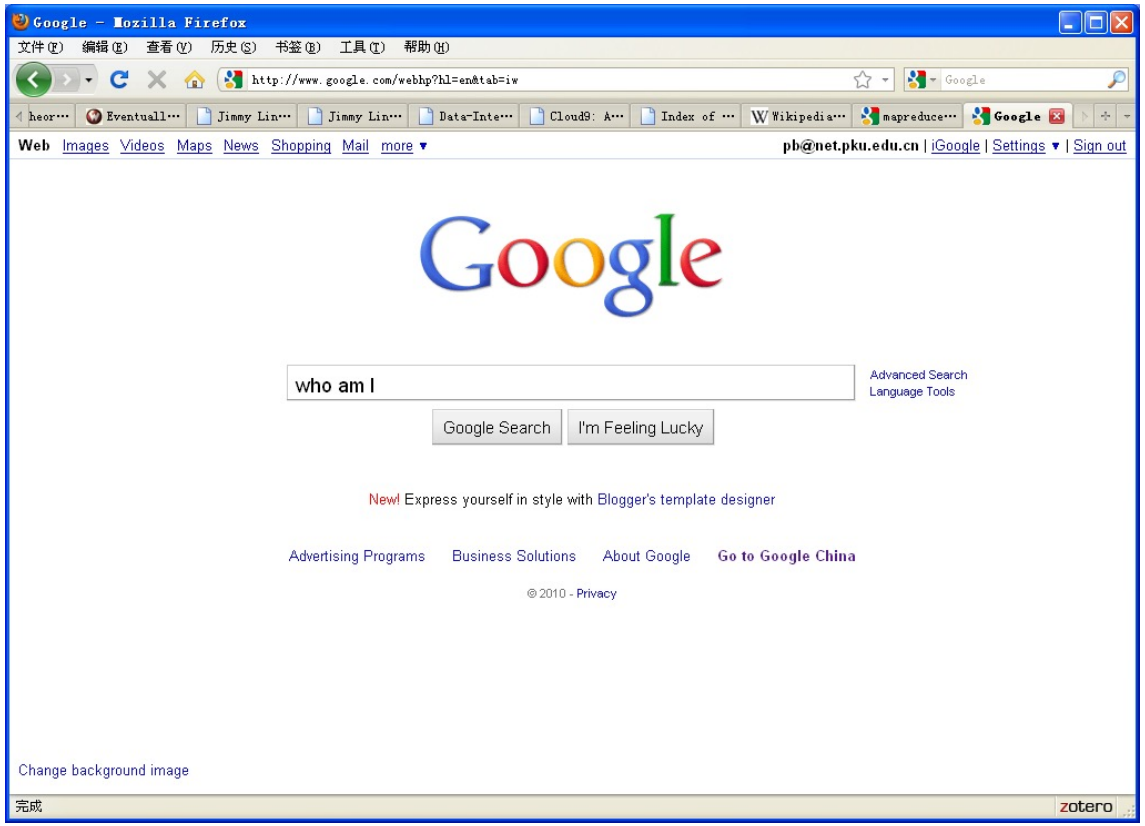

局外人




红楼梦

# 全文搜索引擎算法设计实现

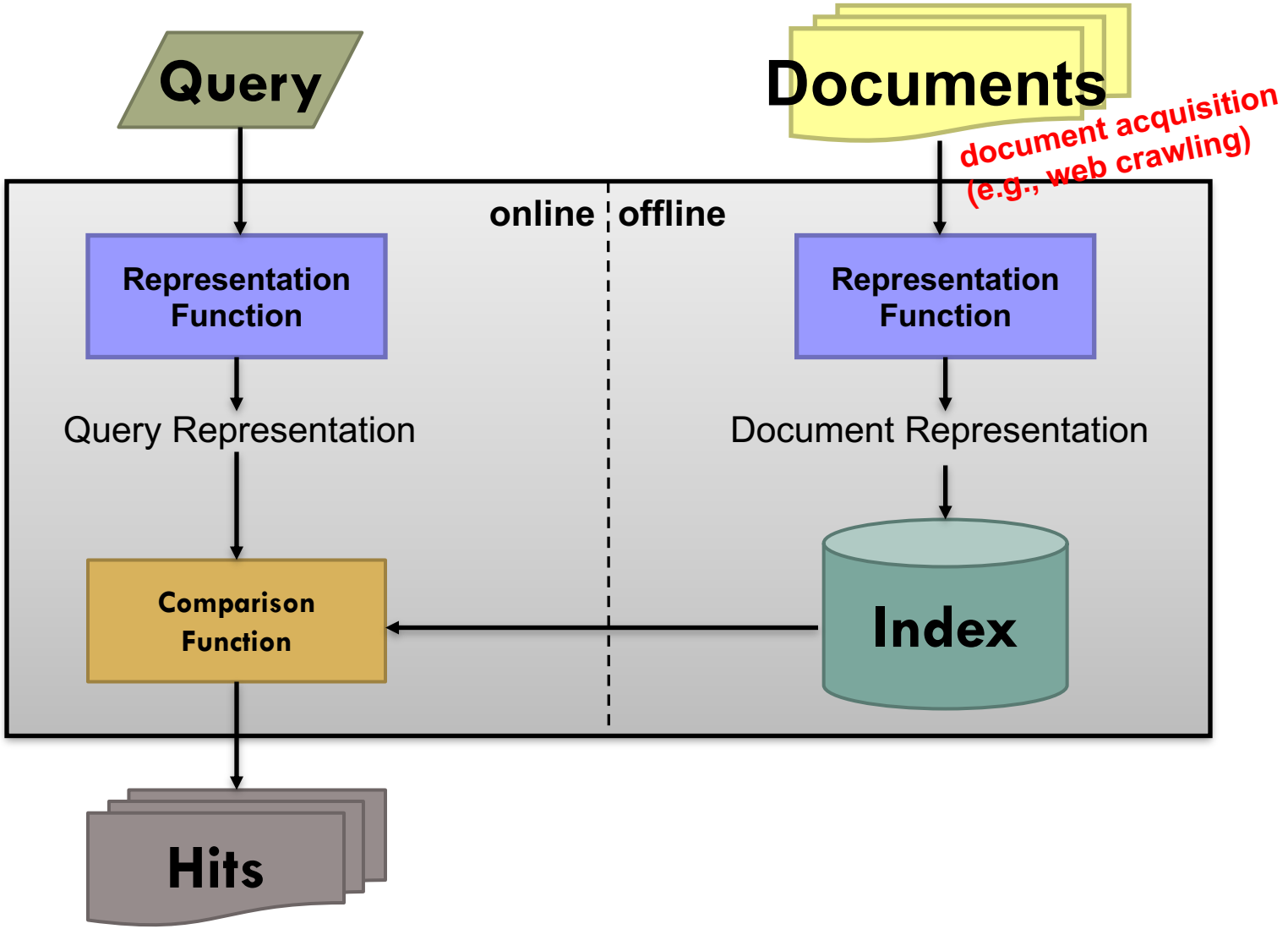





**What happens after you press enter?**

# 体系结构




Query
Documents
document acquisition
(e.g., web crawling)
online
offline
Representation Function
Representation Function
Query Representation
Document Representation
Comparison Function
Index
Hits



# 功能






case folding, tokenization, stopword removal, stemming



sy ax, sem tics, word k wledge, etc.

# 简单的倒排索引示例

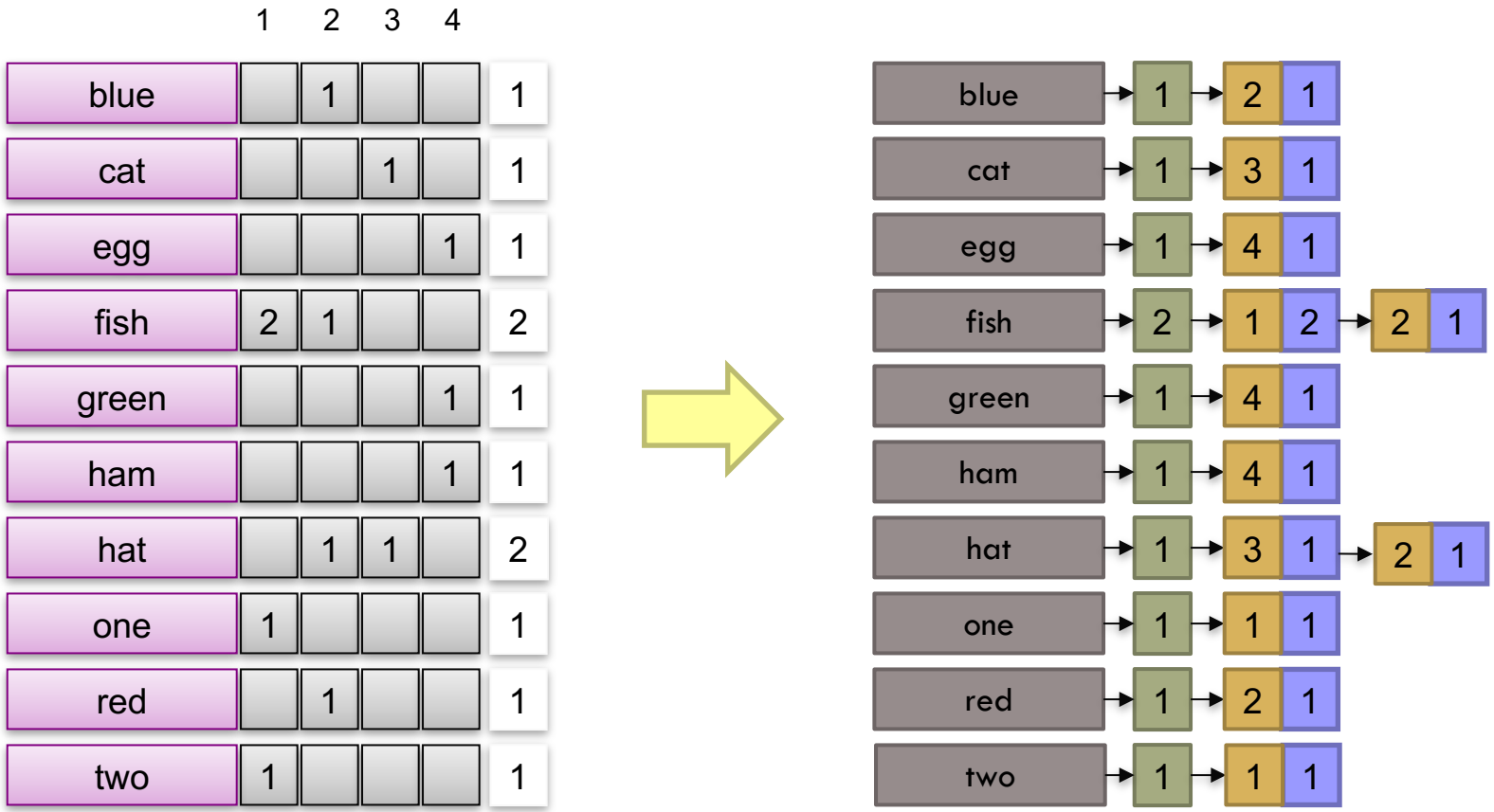


**Doc 1** one fish, two fish

**Doc 2** red fish, blue hat

**Doc 3** cat in the hat

**Doc 4** green eggs and ham





# MapReduce's Role…



| | Character Description | Suitable? |
|---|---|---|
| Indexing Problem | 1.scalability<br>2.relatively fast<br>3.batch operation<br>4.updates may not be important<br>5.crawling is a challenge in itself | Perfect ! |
| Retrieval Problem | 1.must have **sub-second** response time<br>2.for the web, only need relatively **few results** | Not so good … |

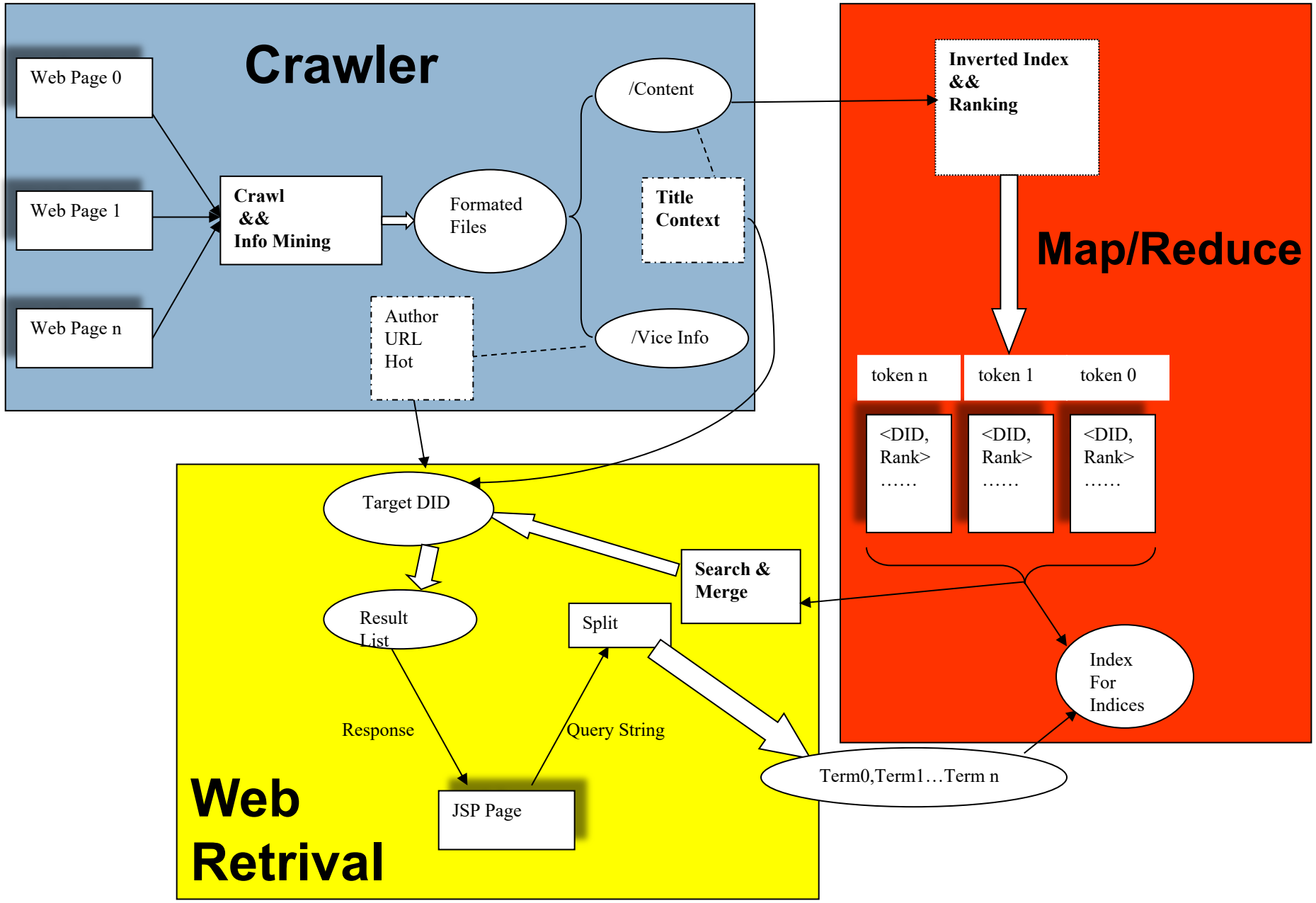


# 小百合全文搜索引擎设计与实现

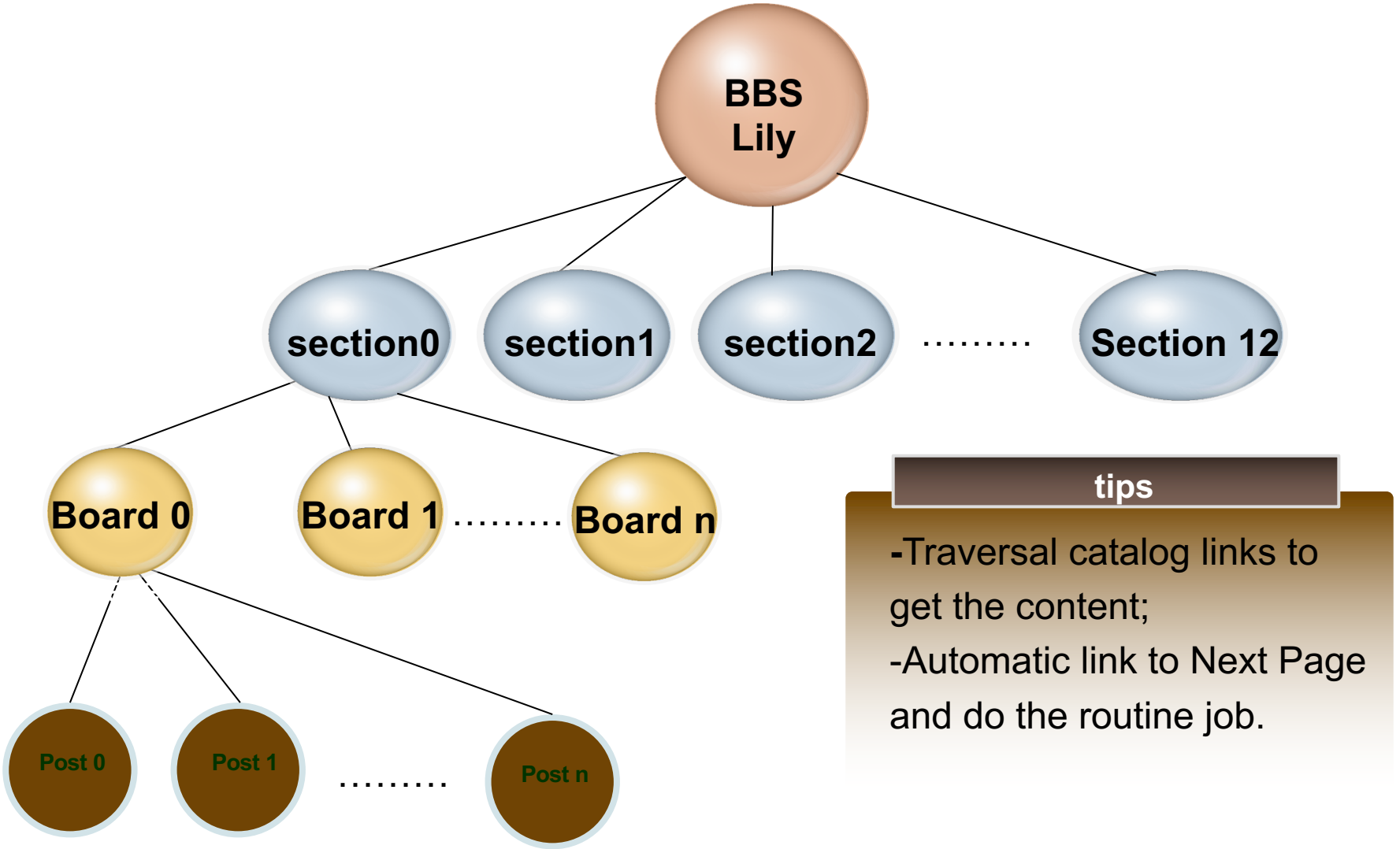


- 工作流程
  - Web page爬取和挖掘
  - 建立索引
  - Web信息检索接口

引自：黄宜华，顾荣 《MapReduce》课程讲义 2011年


Crawler
Web Page 0
Web Page 1
Web Page n
Crawl
&&
Info Mining
Formated
Files
/Content
Title
Context
/Vice Info
Author
URL
Hot
Inverted Index
&&
Ranking
Map/Reduce
token n
token 1
token 0
<DID,
Rank>
……
<DID,
Rank>
……
<DID,
Rank>
……
Index
For
Indices
Target DID
Search &
Merge
Result
List
Split
Response
Query String
JSP Page
Term0,Term1…Term n
Web
Retrival

# Crawler策略： DFS




**tips**

-Traversal catalog links to get the content;

-Automatic link to Next Page and do the routine job.

# Miner策略： Regex



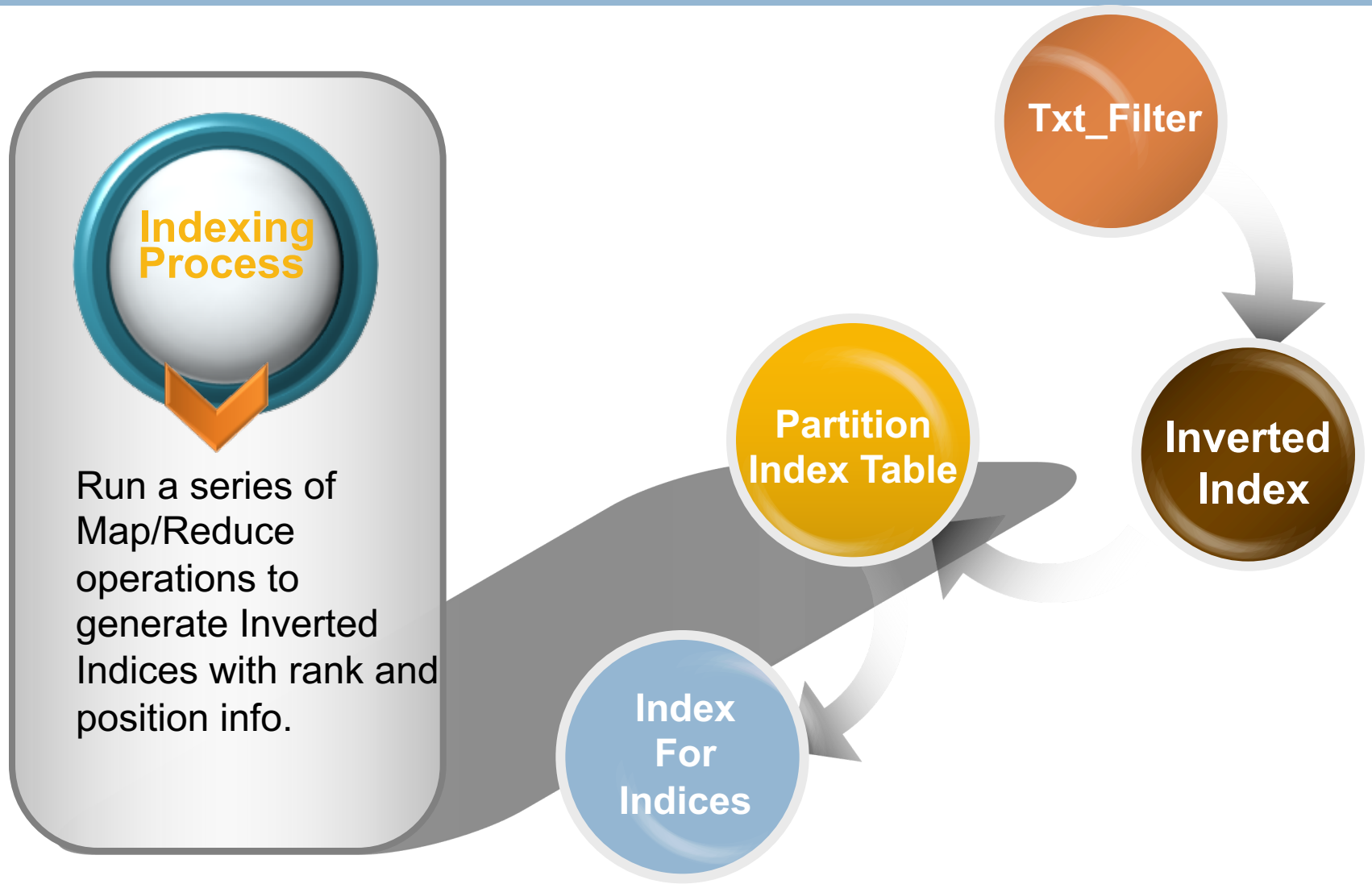



[Each post will be stored in a line as the pattern blew]

**URL'/007'hot'/007'auhtor'/007'title'/007'content**

# 建立索引




Indexing Process
Run a series of Map/Reduce operations to generate Inverted Indices with rank and position info.
Txt_Filter
Inverted Index
Partition Index Table
Index For Indices

# 倒排索引： Posting Info



## Posting Info

**1.TF-IDF (Term Frequency-Inverse Document Frequency):**

$$\mathrm{tf_i} = \frac{n_i}{\sum_k n_k}$$

$$\mathrm{idf_i} = \lg \frac{|D|}{|\{j : t_i \in d_j\}|}$$

$$\mathrm{tfidf} = \mathrm{tf} \cdot \mathrm{idf}$$

- $|D|$ : total number of documents in the corpus
- $|\{d : t_i \in d\}|$: number of documents where the term *ti* appears (that is $n_i \neq 0$ )

**2.Positions info do not need any calculation. They can be record as a Integer Pair like(StartIndex, EndIndex).**

# Target of Web Retrieval Interface






Technics:

- Javascript+HTML
- Tomcat+Servlet

Make an Interface which accept user's query and response search results.

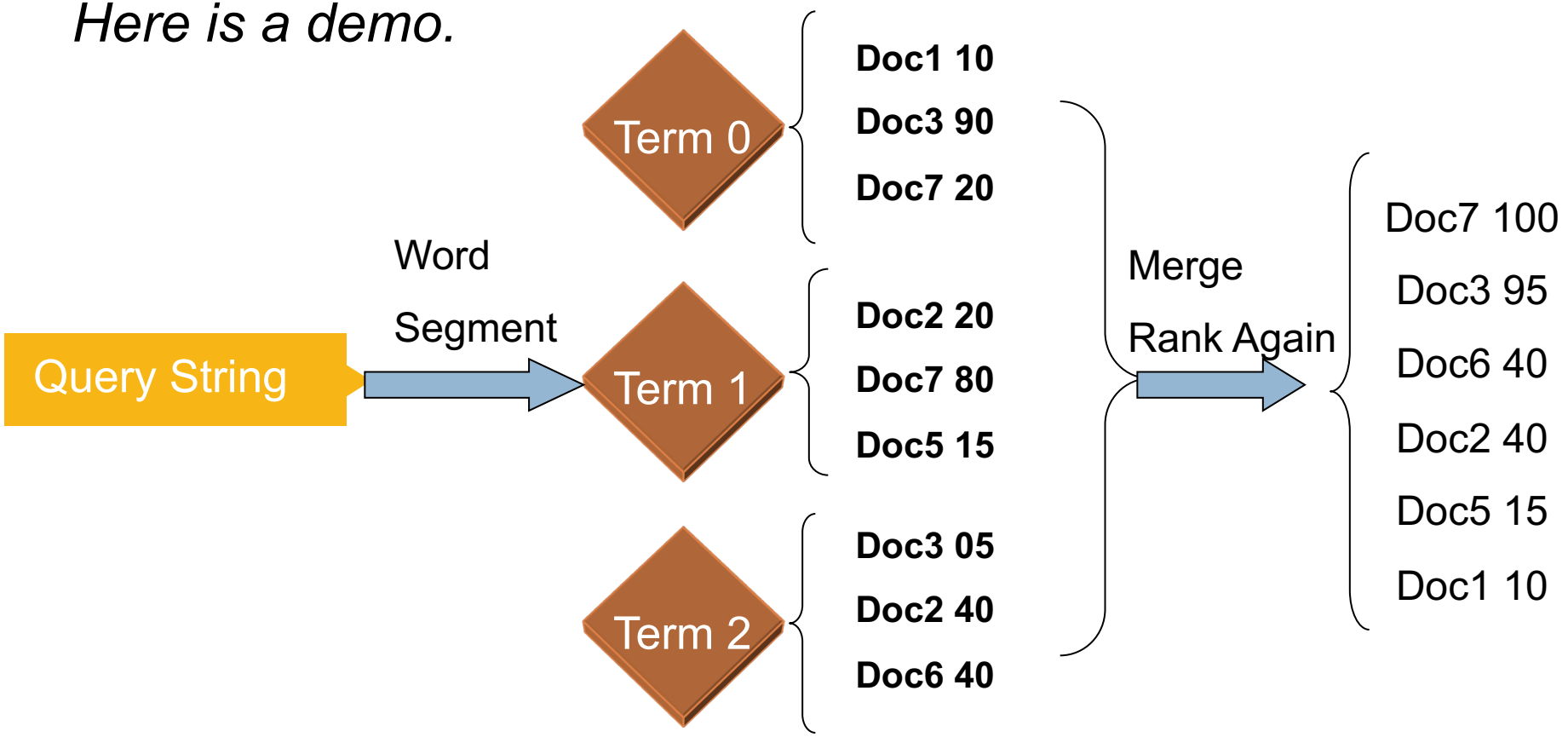
1. **Restrict the query string;**
2. **Sort search result dynamically;**
3. **Response results page by page.**



# Sort Pages



*Here is a demo.*

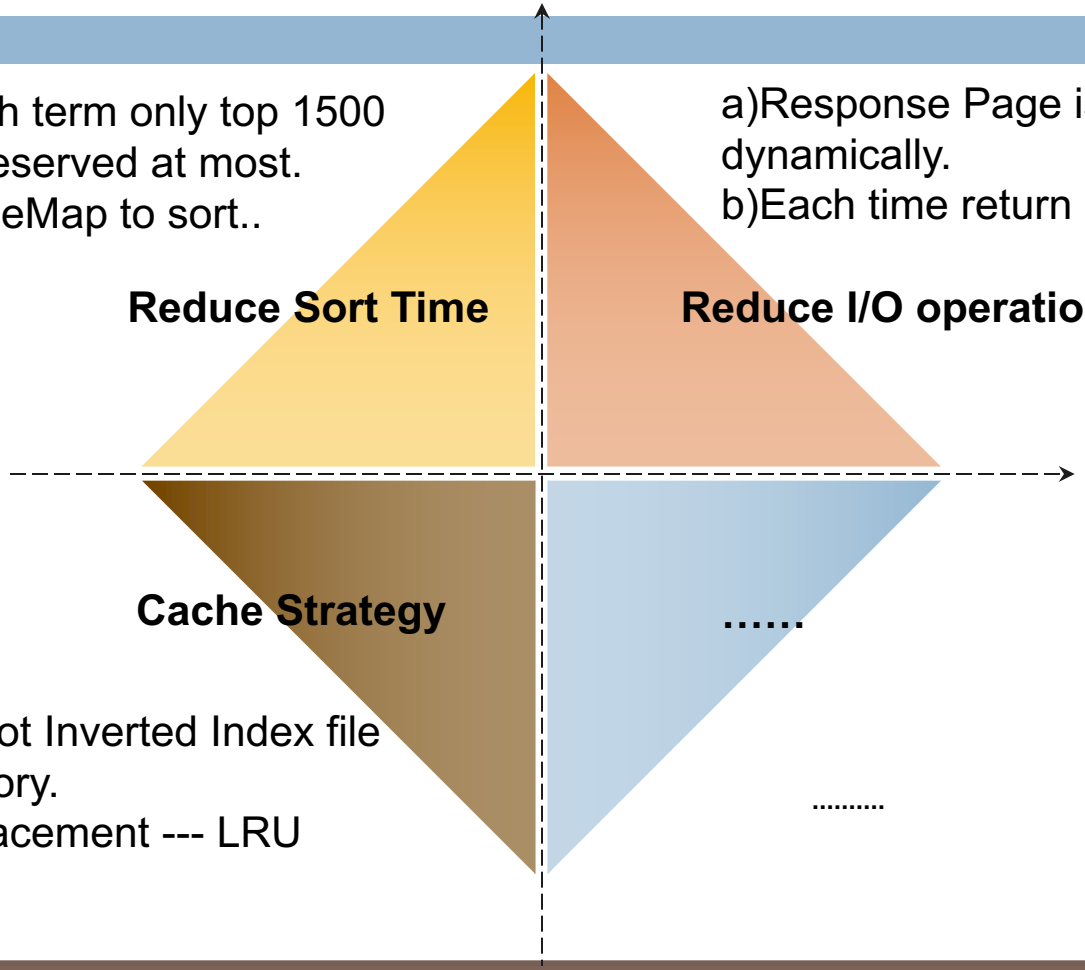


# 优化



a)For each term only top 1500 DID are reserved at most.
b)Use TreeMap to sort..

a)Response Page is created dynamically.
b)Each time return 10 records.




a) Put some hot Inverted Index file in the memory.
b) Cache replacement --- LRU

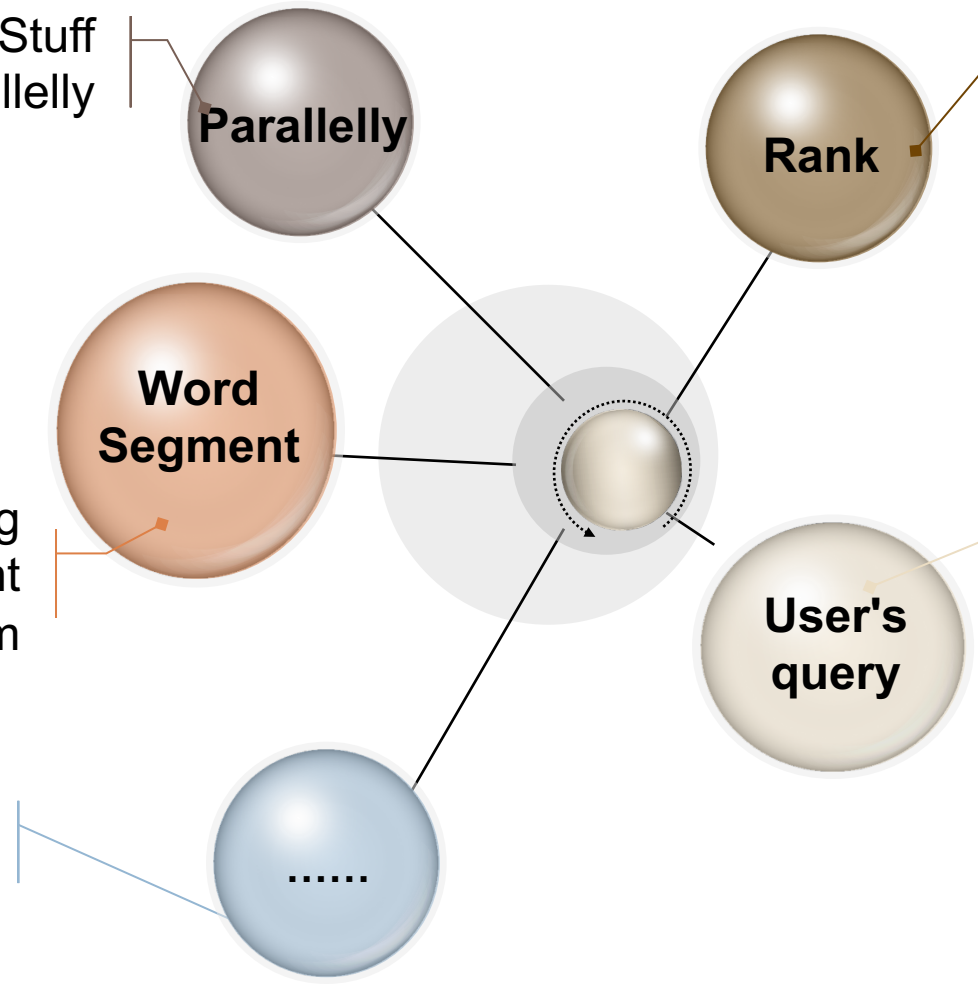
.....

**Optimization measures in different areas.**

# Maybe Google & Baidu has done...



1. Search Stuff parallelly




3.A better rank strategy :To describe the relationship between a token and DID precisely

2. An outstanding Word Segment Algorithm

.....

4.Record each user's query string;
a) feed back to Word Segment
b) Provide remind function.(By input change event)

# 展示



小百合 全文搜索

南京大学小百合联系方式 walk 一下

从30000条帖子中，一共找到最符合搜索要求的1526条结果.

Re: 【提案】一些改善意见 rank:21.362957
发信人: chiyogloria (gloria), 信区: Advice. 点击率: 100
...一些检索点，比如：时间等4.规定“跳蚤”之类的文章规范模式：校区、联系方式、正文等等一些小意见，以后有的话再提，还望好好考虑一下--[1;36
http://bbs.nju.edu.cn/bbscon?board=Advice&file=M.1284596201.A&num=10267

[通告] 招聊天室管理员 rank:16.458197
发信人: danial (流水不争先), 信区: Announce. 点击率: 121
...真实身份和联系方式...
http://bbs.nju.edu.cn/bbscon?board=Announce&file=M.991799473.A&num=1570

[公告]关于学生副斑竹 rank:7.381708
发信人: SYSOP (小百合系统管理员), 信区: Suggestions. 点击率: 149
...询问有关事宜。附各个部门联系方式：学工处 3593154 焦老师团委 ...
http://bbs.nju.edu.cn/bbscon?board=Announce&file=M.985658629.A&num=1197

[公告] 小百合BBS招募站务管理人员 rank:6.3505125
发信人: SYSOP (小百合系统管理员), 信区: Announce. 点击率: 1793
... 如果你有志于对小百合的发展贡献自己的力量，并且有一定的管理经验，富有团队合作和奉献...
... 么申请站务，对百合发展与建设方面的见地等）、申请...
...建设方面的见地等）、申请职务和打算。同时把 真实资料和联系方式提交给站务总管gr...
http://bbs.nju.edu.cn/bbscon?board=Announce&file=M.1082479389.A&num=9156



# 其它MapReduce应用案例



- 大规模基因序列比对
- 大规模城市路径规划
- 大规模重复文档检测
- 基于内容的并行化图像检索算法与引擎
- 大规模微博传播分析
- 城市智能交通综合应用
- ……

# THANK YOU




南京大學
NANJING UNIVERSITY

南京大学计算机软件研究所
Institute of Computer Software, Nanjing University